

Politechnika Wrocławska

Sztuczna Inteligencja i Inżynieria Wiedzy

Raport 3

Planowanie z wykorzystaniem języka PDDL

Dawid Pilarski

Numer indeksu: 280468

Wrocław, 2026

1. Wprowadzenie

Tematem tego zadania było zapoznanie się z językiem PDDL (Planning Domain Definition Language) - językiem deklaratywnym służącym do opisu problemów planowania. W przeciwieństwie do większości języków programowania w PDDL nie pisze się algorytmu rozwiązującego problem, tylko opisuje świat (stany, akcje) i cel, a sekwencję akcji prowadzącą do celu generuje gotowy program zwany planerem.

Lista składa się z trzech zadań o rosnącej złożoności modelowania:

- Zadanie 1 - własny scenariusz transportu paczek z wykorzystaniem rozszerzeń języka (typing, koszty liczbowe, multi-modalność),
- Zadanie 2 - robot odkurzający, gdzie predykaty i akcje są zadane w treści,
- Zadanie 3 - gotowy model PDDL z treści, którego zadaniem jest uruchomienie i analiza wygenerowanego planu.

2. Tło teoretyczne

2.1. Czym jest PDDL

PDDL to język opisu problemów planowania zaproponowany w 1998 roku na potrzeby Międzynarodowych Konkursów Planerów (IPC). Idea jest prosta: oddzielić opis świata od algorytmu szukającego rozwiązania. Programista pisze tylko opis, a planer (gotowy program) sam wymyśla plan.

Każde zadanie PDDL składa się z dwóch plików: `domain.pddl` opisującego reguły świata (jakie obiekty istnieją, jakie akcje są możliwe) i `problem.pddl` opisującego konkretną instancję (jakie konkretnie obiekty mamy, jaki jest stan początkowy i cel).

2.2. Model STRIPS

STRIPS (Stanford Research Institute Problem Solver) to podstawowy model akcji wykorzystywany w PDDL. Każda akcja ma trzy elementy:

- parametry - zmienne, którymi akcja jest sparametryzowana (np. `?from`, `?to`),
- precondition - co musi być prawdą w stanie, żeby akcja była dostępna,
- effect - co zmienia się po wykonaniu akcji (dodawane i usuwane fakty).

2.3. Domena i problem

Domena definiuje typy obiektów (`:types`), predykaty czyli schematy faktów (`:predicates`) oraz schematy akcji. Problem definiuje konkretne obiekty (`:objects`), stan początkowy (`:init`) oraz cel (`:goal`).

Taki podział pozwala raz napisać domenę (np. logistyka) i wielokrotnie ją wykorzystać z różnymi konkretnymi problemami. To trochę jak klasa i jej instancje w programowaniu obiektowym.

2.4. Rozszerzenia PDDL

Czyste STRIPS jest minimalistyczne. Standard PDDL dodaje rozszerzenia, które trzeba zadeklarować w sekcji `:requirements` domeny:

- `:typing` - wprowadza typowanie obiektów i hierarchię typów (np. truck jest podtypem vehicle),
- `:negative-preconditions` - pozwala używać warunków typu "X NIE jest prawdą",
- `:action-costs` - pozwala akcjom mieć liczbowy koszt (`:functions`), a planerowi minimalizować ich sumę,
- `:durative-actions` - pozwala akcjom mieć czas trwania, a planowi być równoległym czasowo.

Im więcej rozszerzeń, tym bogatszy model, ale tym trudniejsze planowanie i mniej planerów obsługuje wszystkie kombinacje.

2.5. Planery i heurystyki

Planer to program rozwiązujący problem planowania. W praktyce planery klasyczne (do STRIPS) sprowadzają planowanie do przeszukiwania grafu stanów, w którym wierzchołki to stany świata, a krawędzie to akcje. Najpopularniejsze podejście to A* z dobrze dobraną heurystyką szacującą koszt z danego stanu do celu.

Najważniejsze heurystyki używane w planowaniu:

- hFF (Fast Forward) - niedopuszczalna, ale w praktyce bardzo skuteczna. Wylicza koszt na relaksacji problemu (ignoruje efekty negatywne).
- hmax - dopuszczalna (gwarantuje optymalność z A*), ale słabsza informacyjnie i wolniejsza.
- hadd - sumuje koszty osiągnięcia każdego subcelu osobno, dopuszczalna tylko w specjalnych przypadkach.

Wybór heurystyki to klasyczny kompromis: hmax daje optymalny plan, ale rozwija dramatycznie więcej węzłów; hFF zwykle znajduje plan suboptymalny o 1-2 akcje, ale w ułamku czasu.

3. Implementacja

3.1. Zadanie 1 - Transport paczek

a) Multi-modalna domena

Zadanie pozwala wykorzystać dowolne rozszerzenia PDDL. Wybrałem scenariusz multi-modalnego transportu, w którym dostępne są trzy równoległe sieci infrastruktury: drogowa, lotnicza i morska. Każdy typ pojazdu może poruszać się wyłącznie po swojej sieci. Domenę zbudowałem na hierarchii typów (truck, plane, ship to podtypy vehicle), dzięki czemu zamiast jednej generycznej akcji move z warunkami typu "jeśli pojazd-jest-ciężarówką" mam trzy osobne akcje: drive, fly, sail, każda przyjmująca konkretny podtyp pojazdu. PDDL pilnuje typowania statycznie, więc nie da się przekazać samolotu do akcji drive.

b) Scenariusz transportowy

W problemie mamy 3 paczki, 4 ciężarówki rozstawione strategicznie, 1 statek (w portA) i 1 samolot (w airportA). Paczki pkg1 i pkg2 startują w warehouse, pkg3 startuje w portA. Cel: wszystkie 3 paczki w shop. Planer ma wybór trasy: morska czy lotnicza, każda wymaga dwóch przeładunków i dwóch przejazdów ciężarówką.

c) Wariant z kosztami liczbowymi

Druga wersja domeny (domain_costs.pddl) dodaje rozszerzenie :action-costs i funkcje numeryczne (road-cost, water-cost, air-cost). Każda akcja ruchu zwiększa total-cost o koszt zależny od pary lokacji, a problem zawiera dyrektywę (:metric minimize (total-cost)). Planer minimalizuje sumę kosztów zamiast liczby akcji. Trasa morska zadana jako tańsza (5) niż lotnicza (30), żeby uchwycić realny kompromis logistyczny.

3.2. Zadanie 2 - Robot odkurzający

Treść zadania podaje predykaty (at, dirty, clean) i listę akcji (move, clean). Implementacja jest prosta. Dwie decyzje projektowe warto odnotować:

- W precondition akcji clean dałem (dirty ?p) - bez tego planer mógłby "czyścić" już czysty pokój i wstawiać do planu bezsensowne akcje.
- Brak predykatu połączeń między pokojami - robot przechodzi z dowolnego pokoju do dowolnego. Treść tego nie precyzuje, a model w zadaniu 3 też używa wolnego ruchu, więc dla spójności tak samo zrobiłem tutaj.

W problemie zdefiniowałem 3 brudne pokoje i robota startującego w pokoj1.

3.3. Zadanie 3 - Robot przenoszący piłki

Domena i problem są podane w treści zadania. Skopiowałem je. Domena modeluje robota z dwoma ramionami poruszającego się między dwoma pokojami. Akcje: move, pick-up (wymaga arm-empty), put-down. Cel: przenieść 4 piłki z room1 do room2.

4. Eksperymenty

4.1. Zadanie 1 - Transport

a) Podstawowy plan (LAMA-first)

Podstawowy run: problem główny w edytorze online z planerem LAMA-first. LAMA-first łączy heurystykę landmark_sum z heurystyką FF i od razu znajduje plan optymalny pod względem długości.

Found Plan (output)

(load pkg2 truck1 warehouse)

(load pkg1 truck1 warehouse)

(drive truck1 warehouse porta)

(unload pkg2 truck1 porta)

(unload pkg1 truck1 porta)

(load pkg3 ship1 porta)

(load pkg1 ship1 porta)

(sail ship1 porta portb)

(unload pkg3 ship1 portb)

(load pkg3 truck2 portb)

(unload pkg2 ship1 portb)

(load pkg2 truck2 portb)

(unload pkg1 ship1 portb)

(load pkg1 truck2 portb)

(drive truck2 portb shop)

(unload pkg3 truck2 shop)

(unload pkg2 truck2 shop)

(unload pkg1 truck2 shop)

```
(:action load
:parameters (pkg2 truck1 warehouse)
:precondition
  (and
    (at-pkg pkg2 warehouse)
    (at-veh truck1 warehouse)
  )
:effect
  (and
    (not
      (at-pkg pkg2 warehouse)
    )
    (in pkg2 truck1)
  )
)
```

```
[t=0.027604s, 12508 KB] Plan length: 19 step(s).
[t=0.027604s, 12508 KB] Plan cost: 19
[t=0.027604s, 12508 KB] Expanded 51 state(s).
[t=0.027604s, 12508 KB] Reopened 0 state(s).
[t=0.027604s, 12508 KB] Evaluated 52 state(s).
[t=0.027604s, 12508 KB] Evaluations: 104
[t=0.027604s, 12508 KB] Generated 566 state(s).
[t=0.027604s, 12508 KB] Dead ends: 0 state(s).
[t=0.027604s, 12508 KB] Number of registered states: 52
[t=0.027604s, 12508 KB] Int hash set load factor: 52/64 = 0.812500
[t=0.027604s, 12508 KB] Int hash set resizes: 6
[t=0.027604s, 12508 KB] Search time: 0.001270s
[t=0.027604s, 12508 KB] Total time: 0.027604s
Solution found.
Peak memory: 12508 KB
Remove intermediate file output.sas
search exit code: 0

INFO    Planner time: 0.37s
```

Plan ma długość 19 akcji i wykorzystuje trasę morską mimo że alternatywa lotnicza też była dostępna. Powód: statek startuje w portA razem z pkg3, więc cały transport można skonsolidować w jednym kierunku bez nadmiarowych przejazdów. Czas obliczeń (0.37s) i liczba rozwiniętych węzłów (51) pokazują, że dla tej skali problemu LAMA-first działa w ułamku sekundy.

b) Eksperyment topologiczny - wariant bez trasy morskiej

Modyfikuję problem usuwając fakty (water-connected portA portB) i (water-connected portB portA) z init (plik problem_no_water.pddl). Reszta bez zmian. Sprawdzam, jak planer zareaguje na zerwanie szlaku morskiego.

Found Plan (output)

(load pkg2 truck1 warehouse)
(load pkg1 truck1 warehouse)
(drive truck1 warehouse portA)
(load pkg3 truck1 portA)
(drive truck1 portA warehouse)
(drive truck1 warehouse airportA)
(unload pkg3 truck1 airportA)
(unload pkg2 truck1 airportA)
(unload pkg1 truck1 airportA)
(load pkg3 plane1 airportA)
(load pkg2 plane1 airportA)
(load pkg1 plane1 airportA)
(fly plane1 airportA airportB)
(unload pkg3 plane1 airportB)
(load pkg3 truck3 airportB)
(unload pkg2 plane1 airportB)
(load pkg2 truck3 airportB)
(unload pkg1 plane1 airportB)
(load pkg1 truck3 airportB)
(drive truck3 airportB shop)
(unload pkg3 truck3 shop)
(unload pkg2 truck3 shop)
(unload pkg1 truck3 shop)

```
(:action load
:parameters (pkg2 truck1 warehouse)
:precondition
  (and
    (at-pkg pkg2 warehouse)
    (at-veh truck1 warehouse)
  )
:effect
  (and
    (not
      (at-pkg pkg2 warehouse)
    )
    (in pkg2 truck1)
  )
)
```

```
unload pkg1 truck3 shop (1)
[t=0.023100s, 12508 KB] Plan length: 23 step(s).
[t=0.023100s, 12508 KB] Plan cost: 23
[t=0.023100s, 12508 KB] Expanded 60 state(s).
[t=0.023100s, 12508 KB] Reopened 0 state(s).
[t=0.023100s, 12508 KB] Evaluated 61 state(s).
[t=0.023100s, 12508 KB] Evaluations: 122
[t=0.023100s, 12508 KB] Generated 603 state(s).
[t=0.023100s, 12508 KB] Dead ends: 0 state(s).
[t=0.023100s, 12508 KB] Number of registered states: 61
[t=0.023100s, 12508 KB] Int hash set load factor: 61/64 = 0.953125
[t=0.023100s, 12508 KB] Int hash set resizes: 6
[t=0.023100s, 12508 KB] Search time: 0.001358s
[t=0.023100s, 12508 KB] Total time: 0.023100s
Solution found.
Peak memory: 12508 KB
Remove intermediate file output.sas
search exit code: 0

INFO      Planner time: 0.34s
```

Plan się wydłużył z 19 do 23 akcji. Powody są dwa:

- Planer musiał przerzucić się ze statku na samolot.
- pkg3 startuje w portA, ale samolot lata między lotniskami. Ciężarówka musi pojechać do portA, zabrać pkg3, wrócić i pojechać do airportA - dwa dodatkowe przejazdy.

Pokazuje to dobrze, że topologia sieci transportowej ma większy wpływ na długość planu niż wybór planera. Zerwanie jednej krawędzi w grafie połączeń (trasa morska) wydłużyło plan o 4 akcje, i nie chodzi tu o gorsze planowanie, tylko o to że alternatywna ścieżka jest fizycznie dłuższa.

c) Wariant z kosztami liczbowymi

Wersja z :action-costs (domain_costs.pddl + problem_costs.pddl) uruchamiana przez planer online (Metric Fast-Forward 2.0). W tym wariancie:

- Koszt jazdy ciężarówką: 8-10 jednostek za odcinek,
- Koszt rejsu statkiem: 5 jednostek (tani),
- Koszt lotu samolotem: 30 jednostek (drogi),
- Koszt load/unload: 1 jednostka.

```

stdout:
((1.00*(TOTAL-COST)) - () + 0.00)
STARTPLANSTARTPLANSTARTPLAN
0: (LOAD PKG1 TRUCK1 WAREHOUSE)
1: (LOAD PKG2 TRUCK1 WAREHOUSE)
2: (DRIVE TRUCK1 WAREHOUSE PORTA)
3: (UNLOAD PKG1 TRUCK1 PORTA)
4: (UNLOAD PKG2 TRUCK1 PORTA)
5: (LOAD PKG1 SHIP1 PORTA)
6: (LOAD PKG2 SHIP1 PORTA)
7: (LOAD PKG3 SHIP1 PORTA)
8: (SAIL SHIP1 PORTA PORTB)
9: (UNLOAD PKG1 SHIP1 PORTB)
10: (UNLOAD PKG2 SHIP1 PORTB)
11: (UNLOAD PKG3 SHIP1 PORTB)
12: (LOAD PKG1 TRUCK2 PORTB)
13: (LOAD PKG2 TRUCK2 PORTB)
14: (LOAD PKG3 TRUCK2 PORTB)
15: (DRIVE TRUCK2 PORTB SHOP)
16: (UNLOAD PKG1 TRUCK2 SHOP)
17: (UNLOAD PKG2 TRUCK2 SHOP)
18: (UNLOAD PKG3 TRUCK2 SHOP)
plan cost: 41.000000
ENDPLANENDPLANENDPLAN

```

```

stderr:

ff: parsing domain file
domain 'TRANSPORT-COSTS' defined
... done.
ff: parsing problem file
problem 'TRANSPORT-3PKG-COSTS' defined
... done.

```

Trasa morska kosztuje 41 jednostek, lotnicza ~65. Różnica wynika głównie z kosztu samego przejazdu między portami/lotniskami (5 vs 30). W wariancie z kosztami planer minimalizuje sumę kosztów, a nie liczbę akcji - to dwa różne kryteria optymalizacji, które przy odpowiednio dobranych kosztach dają różne plany.

4.2. Zadanie 2 - Robot odkurzający

Found Plan (output)

(clean rumba pokoj1)
(move rumba pokoj1 pokoj2)
(clean rumba pokoj2)
(move rumba pokoj2 pokoj3)
(clean rumba pokoj3)

```

(:action clean
:parameters (rumba pokoj1)
:precondition
  (and
    (at rumba pokoj1)
    (dirty pokoj1)
  )
:effect
  (and
    (not
      (dirty pokoj1)
    )
    (clean pokoj1)
  )
)

```

```

[...
[t=0.021588s, 12512 KB] Plan length: 5 step(s).
[t=0.021588s, 12512 KB] Plan cost: 5
[t=0.021588s, 12512 KB] Expanded 8 state(s).
[t=0.021588s, 12512 KB] Reopened 0 state(s).
[t=0.021588s, 12512 KB] Evaluated 9 state(s).
[t=0.021588s, 12512 KB] Evaluations: 18
[t=0.021588s, 12512 KB] Generated 22 state(s).
[t=0.021588s, 12512 KB] Dead ends: 0 state(s).
[t=0.021588s, 12512 KB] Number of registered states: 9
[t=0.021588s, 12512 KB] Int hash set load factor: 9/16 = 0.562500
[t=0.021588s, 12512 KB] Int hash set resizes: 4
[t=0.021588s, 12512 KB] Search time: 0.000374s
[t=0.021588s, 12512 KB] Total time: 0.021588s
Solution found.
Peak memory: 12512 KB
Remove intermediate file output.sas
search exit code: 0

INFO    Planner time: 0.34s

```

Plan zawiera 5 akcji = 3× clean + 2× move. To plan optymalny - niżej zejść się nie da, bo każdy pokój wymaga osobnej akcji clean (3 akcje, nieuniknione), a robot ma jedną pozycję na raz i musi się przemieścić do każdego pokoju (minimum 2 ruchy startując z pokoj1).

Razem 3 + 2 = 5. Kolejność wybrana przez planer (pokoj1 → pokoj3 → pokoj2) jest arbitralna - bez kosztów drogi obie alternatywne kolejności są równie dobre, planer wybiera pierwszą znaną.

4.3. Zadanie 3 - Robot z piłkami

Found Plan (output)

(pick-up robot arm1 ball1 room1)

(pick-up robot arm2 ball2 room1)

(move robot room1 room2)

(put-down robot arm1 ball1 room2)

(put-down robot arm2 ball2 room2)

(move robot room2 room1)

(pick-up robot arm1 ball3 room1)

(pick-up robot arm2 ball4 room1)

(move robot room1 room2)

(put-down robot arm1 ball3 room2)

(put-down robot arm2 ball4 room2)

```

(:action pick-up
:parameters (robot arm1 ball1 room1)
:precondition
  (and
    (at robot room1)
    (inroom ball1 room1)
    (arm-empty arm1)
  )
:effect
  (and
    (holding arm1 ball1)
    (not
      (arm-empty arm1)
    )
    (not
      (inroom ball1 room1)
    )
  )
)

```

```

[t=0.026605s, 12508 KB] Plan length: 11 step(s).
[t=0.026605s, 12508 KB] Plan cost: 11
[t=0.026605s, 12508 KB] Expanded 16 state(s).
[t=0.026605s, 12508 KB] Reopened 0 state(s).
[t=0.026605s, 12508 KB] Evaluated 17 state(s).
[t=0.026605s, 12508 KB] Evaluations: 34
[t=0.026605s, 12508 KB] Generated 63 state(s).
[t=0.026605s, 12508 KB] Dead ends: 0 state(s).
[t=0.026605s, 12508 KB] Number of registered states: 17
[t=0.026605s, 12508 KB] Int hash set load factor: 17/32 = 0.531250
[t=0.026605s, 12508 KB] Int hash set resizes: 5
[t=0.026605s, 12508 KB] Search time: 0.000525s
[t=0.026605s, 12508 KB] Total time: 0.026605s
Solution found.

Peak memory: 12508 KB
Remove intermediate file output.sas
search exit code: 0

INFO    Planner time: 0.37s

```

Plan ma 11 akcji i logiczną strukturę dwóch wycieczek:

- Wycieczka 1 (akcje 1-5): pick-up dwóch piłek, move do room2, put-down obu piłek.
- Powrót do room1 (akcja 6).
- Wycieczka 2 (akcje 7-11): jak wycieczka 1 dla pozostałych piłek.

Liczba akcji: $2 \times (2 \text{ pick} + 1 \text{ move} + 2 \text{ put}) + 1 \text{ powrót} = 2 \times 5 + 1 = 11$. To plan optymalny - robot ma 2 ramiona, więc maksymalnie przenosi 2 piłki na raz, 4 piłki = minimum 2 wycieczki.

Można też przeanalizować wpływ liczby ramion na długość planu (analiza teoretyczna):

Liczba ramion	Wycieczek	Długość planu
1	4	15
2	2	11
4	1	9

Wzrost liczby ramion daje malejący zysk - z 1 na 2 ramiona zysk to 4 akcje, z 2 na 4 już tylko 2 akcje. To efekt tego, że każda wycieczka ma stały narzut (1 ruch tam) niezależny od liczby przenoszonych piłek.

5. Użyte narzędzia

Projekt używa wyłącznie istniejących planerów PDDL bez dodatkowego kodu po stronie użytkownika:

- editor.planning.domains - webowy edytor PDDL z dostępem do wielu planerów uruchamianych w chmurze. To główne narzędzie, którego używałem do wszystkich zadań. Nie wymaga lokalnej instalacji.
- LAMA-first - konfiguracja planera (heurystyka landmark_sum + ff, search: lazy_greedy) używana dla podstawowych wariantów zadań.
- Metric Fast-Forward 2.0 - wariant planera obsługujący numeric fluents i :action-costs, używany dla wersji z kosztami liczbowymi (zadanie 1, plik domain_costs.pddl).

Obsługa edytora online sprowadza się do trzech kroków:

- Wgranie plików (domain.pddl + problem.pddl) przez Import.
- Kliknięcie Solver i wybór planera z listy.
- Odczytanie planu z panelu "Found Plan" oraz statystyk z "Solver Output" (plan length, plan cost, expanded states, planner time).

6. Wnioski

PDDL jako język opisu problemów planowania radzi sobie dobrze z dekompozycją: oddzielenie domeny (reguł świata) od problemu (konkretnej instancji) pozwala raz napisany model wielokrotnie wykorzystać. Hierarchia typów (:typing) bardzo się przydała przy modelowaniu multi-modalności w zadaniu 1, bo zamiast jednej akcji z warunkami typu "jeśli pojazd-to-truck" można zdefiniować trzy oddzielne, czyste akcje sprawdzające typ statycznie.

LAMA-first poradził sobie ze wszystkimi zadaniami w ułamku sekundy (planner time poniżej 1 sekundy). Dla tej skali zadań nie było sensu porównywać heurystyk ani strategii wyszukiwania, bo domyślne ustawienia dawały od razu optymalne plany. Przy większych problemach (dziesiątki obiektów, setki akcji) wybór heurystyki i sortowanie ruchów miałyby duże znaczenie.

Eksperyment topologiczny (zadanie 1, wariant bez wody) potwierdza, że modelowanie świata ma większy wpływ na koszt planu niż dobór algorytmu. Zerwanie jednego połączenia w sieci wydłużyło plan z 19 do 23 akcji (o 21%). Praktycznie to ważna obserwacja, bo projektowanie infrastruktury (gdzie postawić port, lotnisko) jest decyzją grubszą niż wybór planera.

Ostatnia rzecz: wersja z :action-costs pokazuje, że minimalizacja długości planu i minimalizacja sumy kosztów to różne kryteria. W zadaniu 1 trasa morska i lotnicza mają (przy odpowiednim ustawieniu pojazdów) podobną liczbę akcji, ale różne koszty: 41 vs ~65 jednostek. Bez rozszerzenia o koszty liczbowe model nie złapałby realnego kompromisu logistycznego "tani statek vs drogi samolot".

7. Materiały źródłowe

1. PDDL Reference, <https://planning.wiki/ref/pddl/domain>
2. Fast Downward planning system, <https://www.fast-downward.org/>
3. Editor.planning.domains, <https://editor.planning.domains/>
4. Materiały wykładowe SIiIW, Politechnika Wrocławska, 2026.
5. Gemini Pro